C++ COMPLETE REFERENCE LIBRARY • MODULE 22

# 22 PROGRAMMING PRACTICE — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

## Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 mathematical computing problem solving
3. **Theory Chapter:** 02 oop design patterns for exams
4. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
5. **Annotated C++ Source Programs** (4 Programs)
6. **Practice Question Bank:** programming questions
7. **Step-by-Step Solutions:** programming solutions

## 22 — B.Tech Mathematics and Computing Programming Practice

This module contains integrated mathematical and object-oriented problem-solving exercises tailored for a B.Tech Mathematics and Computing curriculum.

### Contents

- `01_THEORY/01_mathematical_computing_problem_solving.md` (./01_THEORY/01_mathematical_computing_problem_solving.md)
- `01_THEORY/02_oop_design_patterns_for_exams.md` (./01_THEORY/02_oop_design_patterns_for_exams.md)
- `02_PROGRAMS/01_polynomial_addition_class.cpp` (./02_PROGRAMS/01_polynomial_addition_class.cpp)
- `02_PROGRAMS/02_rational_fraction_class.cpp` (./02_PROGRAMS/02_rational_fraction_class.cpp)
- `02_PROGRAMS/03_matrix_multiplication_class.cpp` (./02_PROGRAMS/03_matrix_multiplication_class.cpp)
- `02_PROGRAMS/04_binary_search_pointer_range.cpp` (./02_PROGRAMS/04_binary_search_pointer_range.cpp)
- `03_PRACTICE_QUESTIONS/programming_questions.md` (./03_PRACTICE_QUESTIONS/programming_questions.md)
- `04_DETAILED_SOLUTIONS/programming_solutions.md` (./04_DETAILED_SOLUTIONS/programming_solutions.md)

- `05_DIAGRAMS/diagram_01_polynomial_addition.svg`
  (./05_DIAGRAMS/diagram_01_polynomial_addition.svg)

# 01 — Mathematical Computing Problem-Solving in C++23

**Component-Wise Addition of Quadratic Polynomials**

```
p1 = 3x^2 + 4x + 5 | p2 = 2x^2 + 6x + 1

p1 + p2 = (3+2)x^2 + (4+6)x + (5+1) = 5x^2 + 10x + 6
```

**Polynomial Addition**

In B.Tech Mathematics and Computing examinations, classic mathematical structures — **Quadratic Polynomials**, **Rational Fractions ($p/q$)**, **Complex Numbers ($a+bi$)**, and **Matrices** — are modeled as simple classes with constructors, `operator+`, `operator*`, and `display()` member functions.

# 02 — Exam Strategy: How to Structure Any University OOP Question

1. **Identify Data Members (`private:` or `protected:`)**: Keep state minimal (e.g., `int num, den;` for fractions).
2. **Provide a Default + Parameterized Constructor**: `fraction(int n = 0, int d = 1)`.
3. **Write Clean Member Functions (`input()`, `display()`)** and required operator overloads (`operator+`).
4. **Demonstrate in `int main()`**: Create 2 objects, perform the operation, and call `display()`.

# Visual Concept & Memory Diagrams (22 PROGRAMMING PRACTICE)

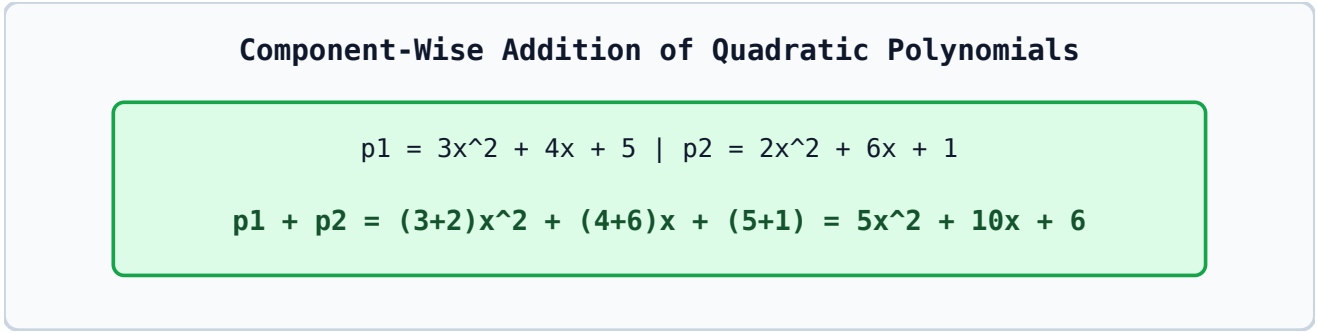



**Diagram: diagram 01 polynomial addition**

# Annotated C++ Programs (22 PROGRAMMING PRACTICE)

## Program: `01_polynomial_addition_class.cpp`

**C++ SOURCE FILE • 01_polynomial_addition_class.cpp**

```cpp
// Program Title: Quadratic Polynomial Addition Using Operator Overloading
// Topic: Programming Practice — Mathematical Computing with Classes
// Purpose: Add two quadratic polynomials (ax^2 + bx + c) using operator+.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_mathematical_computing_problem_solving.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class polynomial {
private:
    int a, b, c; // Represents a*x^2 + b*x + c

public:
    polynomial(int c2 = 0, int c1 = 0, int c0 = 0) {
        a = c2;
        b = c1;
        c = c0;
    }

    polynomial operator+(const polynomial& p) const {
        return polynomial(a + p.a, b + p.b, c + p.c);
    }

    void display() const {
        cout << a << "x^2 + " << b << "x + " << c << endl;
    }
};

int main() {
    polynomial p1(3, 4, 5);
    polynomial p2(2, 6, 1);
    polynomial p3 = p1 + p2;

    p3.display();
    return 0;
}

/*
Expected Deterministic Output:
5x^2 + 10x + 6
*/
```

## Program: `02_rational_fraction_class.cpp`

**C++ SOURCE FILE • 02_rational_fraction_class.cpp**

```cpp
// Program Title: Rational Number (p/q) Addition in Reduced Form
// Topic: Programming Practice — Rational Arithmetic
// Purpose: Add two fractions and reduce them using greatest common divisor (GCD).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_mathematical_computing_problem_solving.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class fraction {
private:
    int num;
    int den;

    static int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }

public:
    fraction(int n = 0, int d = 1) {
        int g = gcd(n, d);
        num = n / g;
        den = d / g;
    }

    fraction operator+(const fraction& f) const {
        return fraction((num * f.den) + (f.num * den), den * f.den);
    }

    void display() const {
        cout << num << "/" << den << endl;
    }
};

int main() {
    fraction f1(1, 4);
    fraction f2(1, 4);
    fraction sum = f1 + f2; // 2/4 → reduced to 1/2
    sum.display();
    return 0;
}

/*
Expected Deterministic Output:
1/2
*/
```

## Program: `03_matrix_multiplication_class.cpp`

**C++ SOURCE FILE • 03_matrix_multiplication_class.cpp**

```cpp
// Program Title: 2x2 Matrix Multiplication Class
// Topic: Programming Practice — Linear Algebra in C++
// Purpose: Multiply two 2x2 matrices using a classroom matrix class.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_mathematical_computing_problem_solving.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class matrix2x2 {
public:
    int m[2][2];

    matrix2x2(int a = 0, int b = 0, int c = 0, int d = 0) {
        m[0][0] = a; m[0][1] = b;
        m[1][0] = c; m[1][1] = d;
    }

    matrix2x2 operator*(const matrix2x2& other) const {
        matrix2x2 res;
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                res.m[i][j] = 0;
                for (int k = 0; k < 2; k++) {
                    res.m[i][j] += m[i][k] * other.m[k][j];
                }
            }
        }
        return res;
    }

    void display() const {
        cout << m[0][0] << " " << m[0][1] << endl;
        cout << m[1][0] << " " << m[1][1] << endl;
    }
};

int main() {
    matrix2x2 A(1, 2, 3, 4);
    matrix2x2 B(2, 0, 1, 2);
    matrix2x2 C = A * B;
    C.display();
    return 0;
}

/*
Expected Deterministic Output:
4 4
10 8
*/
```

## Program: `04_binary_search_pointer_range.cpp`

**C++ SOURCE FILE • 04_binary_search_pointer_range.cpp**

```cpp
// Program Title: Binary Search Using Pointer Arithmetic
// Topic: Programming Practice — Pointers & Searching
// Purpose: Perform binary search on a sorted array using pointers.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_mathematical_computing_problem_solving.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int binary_search_ptr(const int* arr, int n, int target) {
    const int* low = arr;
    const int* high = arr + (n - 1);

    while (low <= high) {
        const int* mid = low + (high - low) / 2;
        if (*mid == target) {
            return static_cast<int>(mid - arr);
        } else if (*mid < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main() {
    int arr[6] = {5, 12, 19, 26, 33, 40};
    cout << "Index of 26: " << binary_search_ptr(arr, 6, 26) << endl;
    return 0;
}

/*
Expected Deterministic Output:
Index of 26: 3
*/
```

# Programming Practice Questions — Module 22

**Q1.** Write a C++23 class `vector3d` with `x, y, z` components that computes the dot product (`operator*`) and vector sum (`operator+`) of two 3D vectors.

# Detailed Solutions — Module 22

CPP

```cpp
#include <iostream>
using namespace std;
class vector3d {
    int x, y, z;
public:
    vector3d(int a = 0, int b = 0, int c = 0) : x(a), y(b), z(c) {}
    int operator*(const vector3d& v) const { return x*v.x + y*v.y + z*v.z; }
};
int main() {
    vector3d u(1, 2, 3), v(4, 5, 6);
    cout << "Dot product: " << (u * v) << endl;
    return 0;
}
// Expected Output: Dot product: 32
```